

Revision 14-17 • The KV Cache Story

Interactive revision of lessons 14, 15, 16 and 17: each question as asked, the answer, where I slipped, and the one-line takeaway.

Q1 • How much work does the naive loop waste? PARTIAL

Covers: 14

QUESTION

A naive loop reruns the model over the whole sequence each step. With a 512-token prompt and a 256-token reply: total tokens processed, tokens actually needed, waste ratio, and why the recomputation is unnecessary?

How to solve it

1. Step k processes $512 + k$ tokens, for $k = 0 \dots 255$.
2. Prompt re-run every step: $N \times P = 256 \times 512 = \mathbf{131,072}$.
3. Growing reply: $0 + 1 + \dots + 255 = N(N - 1)/2 = \mathbf{32,640}$.
4. Total $\approx \mathbf{163,712}$.
5. Needed: every token through the model **once** (to produce its K and V): $P + N = \mathbf{768}$.
6. Waste ratio: $163,712 \div 768 \approx \mathbf{213\times}$ (roughly the reply length).
7. **Why unnecessary:** under the causal mask each position depends only on tokens up to itself, so its numbers are identical every step. Compute once, keep K and V: the KV cache.

RE-DRILL

Separate "what a step uses" from "what must be computed at least once".

Q2 • Read prefill throughput off a TTFT sweep MISSED

Covers: 15

QUESTION

TTFT (one request, max_tokens = 1, medians): 256 tokens $\rightarrow$ 23 ms; 8,000 tokens $\rightarrow$ 280 ms.
Assuming TTFT = fixed cost + tokens $\div$ throughput, what are the prefill throughput and the fixed cost? What is the fixed cost made of?

How to solve it

1. **Slope** = extra time $\div$ extra tokens = $(280 - 23) \text{ ms} \div (8,000 - 256) = 257 \text{ ms} \div 7,744 \approx \mathbf{0.033 \text{ ms/token}}$.

2. **Throughput** = $1 \div 0.033 \text{ ms} \approx 30,000 \text{ tokens/s}$, matching the napkin "half of H100 peak" (lesson 15).
3. **Fixed cost** = $23 - 256 \times 0.033 \approx 23 - 8.5 \approx 14.5 \text{ ms}$.
4. **Made of:** per-request overhead (**networking, tokenization, scheduling**), not prefill (that's the slope) and not decode ($\text{max_tokens} = 1$).

RE-DRILL

With a fixed cost in play, use the difference between two points, never a single point's ratio.

Q3 · TPOT from timestamps, and a stuttering stream PARTIAL

Covers: 16, 15

QUESTION

One request: TTFT 0.3 s, E2E 2.3 s, 401 output tokens. What's TPOT, and why not $\text{E2E} \div 401$? Average TPOT looks fine but users say the stream stutters: what do you look at?

Answer

- **TPOT** = $(\text{E2E} - \text{TTFT}) \div (\text{N}_{\text{out}} - 1) = (2.3 - 0.3) \div 400 = 5 \text{ ms}$. $\text{E2E} \div 401$ would fold the prefill pause (TTFT) into the per-token pace; and the first token arrived with TTFT, so only 400 gaps remain.
- **Stutter:** look at the **individual inter-token gaps** and their tail (P95/P99), not the average. A common cause: another user's big prefill occupies the GPU and every stream pauses behind it (lesson 15); fix with chunked prefill.

Q4 · Three KV cache bugs MISSED

Covers: 17, 11

QUESTION

A preallocated KV cache shows: (1) first token fine, the rest gibberish; (2) outputs subtly worse, buffer preallocated with zeros and attention reads all of it; (3) an answer mentions the previous user's conversation. The bug behind each?

Answer

1. **RoPE position:** queries and keys are rotated by their position. The decode token was fed as position 0 instead of the pointer's position (e.g. 512). The first token comes from prefill (positions right), every later step uses the wrong rotation → gibberish.
2. **Reading empty slots:** a zero key scores $q \cdot 0 = 0$, and $e^0 = 1$, so every unfilled slot steals attention weight. Fix: slice or mask to the filled length.

3. **No reset between requests:** the pointer and old K/V weren't cleared, so the new user attends to the previous user's text. Fix: reset per request. A privacy bug, not just quality.

RE-DRILL

Match the symptom's timing: "first token fine, then broken" points at something that differs between prefill and decode.

Exercise · Toy KV cache vs naive recompute SOLID

Covers: 14, 17

QUESTION

One attention head on random 8-number vectors. Write the cached loop (preallocated K/V buffers, a position pointer, read only filled slots) and prove it matches the naive loop exactly; then read the empty slots to see bug 2.

```
def cached_outputs(x, prompt_len, read_all_slots=False):
    K_cache = np.zeros((max_len, d))          # 1 · preallocated, fixed shape, zeros
    V_cache = np.zeros((max_len, d))
    pos = 0
    K_cache[:prompt_len] = x[:prompt_len] @ Wk  # 2 · prefill: whole prompt in one shot
    V_cache[:prompt_len] = x[:prompt_len] @ Wv
    pos = prompt_len

    def read():                                # 4 · filled rows only (bug flag: all rows)
        n = max_len if read_all_slots else pos
        return K_cache[:n], V_cache[:n]

    outs = [attend(x[prompt_len - 1] @ Wq, *read())] # first output comes from prefill
    for t in range(prompt_len, len(x)):           # 3 · decode: one k, v per new token
        K_cache[pos] = x[t] @ Wk
        V_cache[pos] = x[t] @ Wv
        pos += 1
        outs.append(attend(x[t] @ Wq, *read()))    # query used once, never cached
    return np.array(outs)
```

Result: `cached == naive: True` ; reading the zero-filled slots gives a max error of **~3.97**, because each zero key scores 0 and gets weight $e^0 = 1$.